

Capstone Project

Name: Etor Wisdom Ayomikun

ID: ALT/SOE/025/3807

Course: Altschool Cybersecurity SEAC (Ethical Hacking)

Date: 25/05/2026

Project 1: Internal Employee Portal Audit (Altschool case)

INTRODUCTION

This assessment was conducted using the PortSwigger Web Security Academy lab titled "User ID controlled by request parameter with password disclosure."

The objective of this assessment was to identify and exploit an access control vulnerability, determine its impact, and provide appropriate remediation recommendations.

Task 1: Lab Setup and Reconnaissance

Target Application

The target application was a simulated web application provided by PortSwigger Web Security Academy.

Authentication Status

The application required authentication before users could access account information.

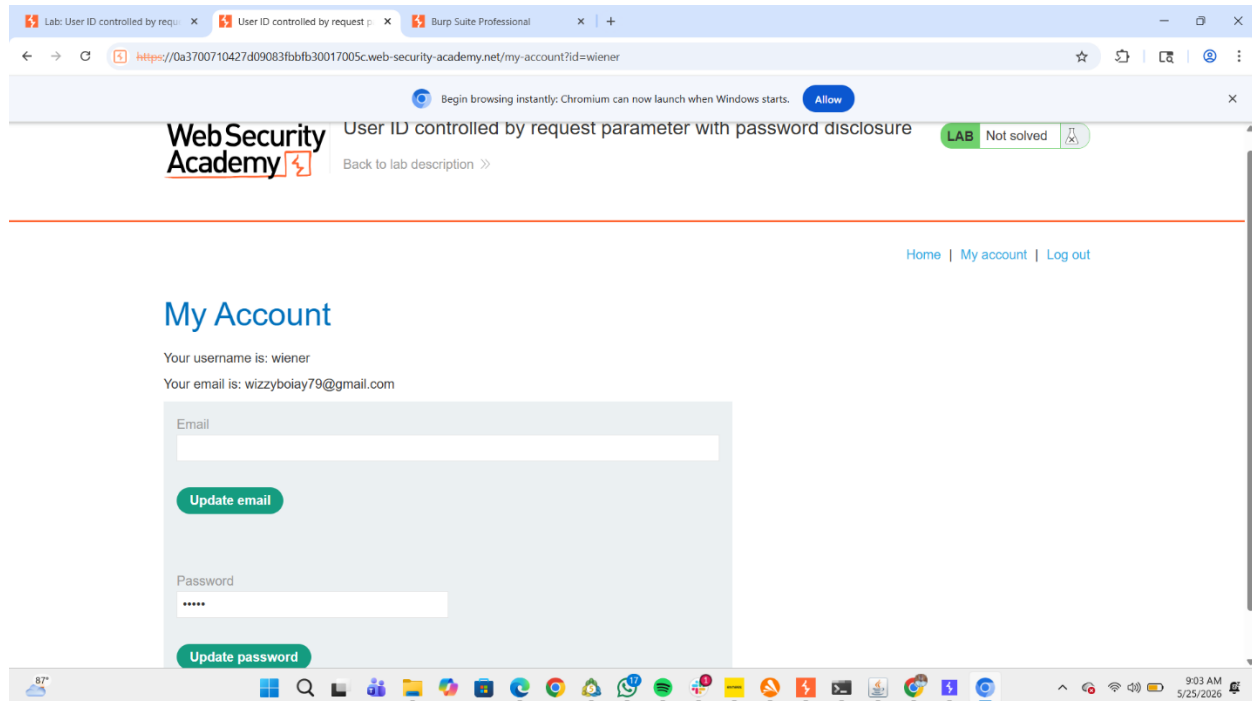
The provided credentials were used to log into the application as a standard user.

Initial Observations

After successful authentication, the application redirected the user to the account page.

The account page URL contained a user-controlled parameter: /my-account?id=wiener

The presence of the **id** parameter suggested that the application used client-supplied input to determine which user's account information would be displayed.



Task 2: Vulnerability Identification

Vulnerable Request

Using Burp Suite, the following request was identified:

GET /my-account?id=wiener HTTP/2

The application relied on the value of the **id** parameter to determine which account data should be returned.

No authorization checks were performed to verify whether the currently authenticated user was permitted to access the requested account.

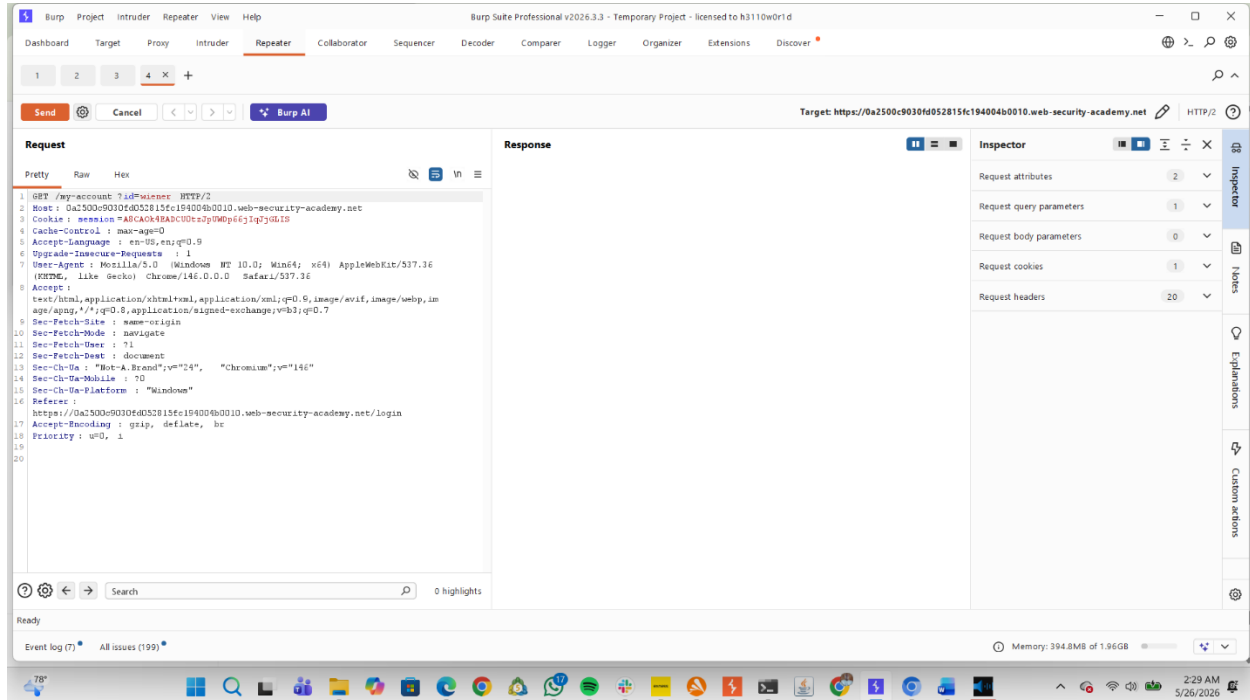
Vulnerability Type

Broken Access Control

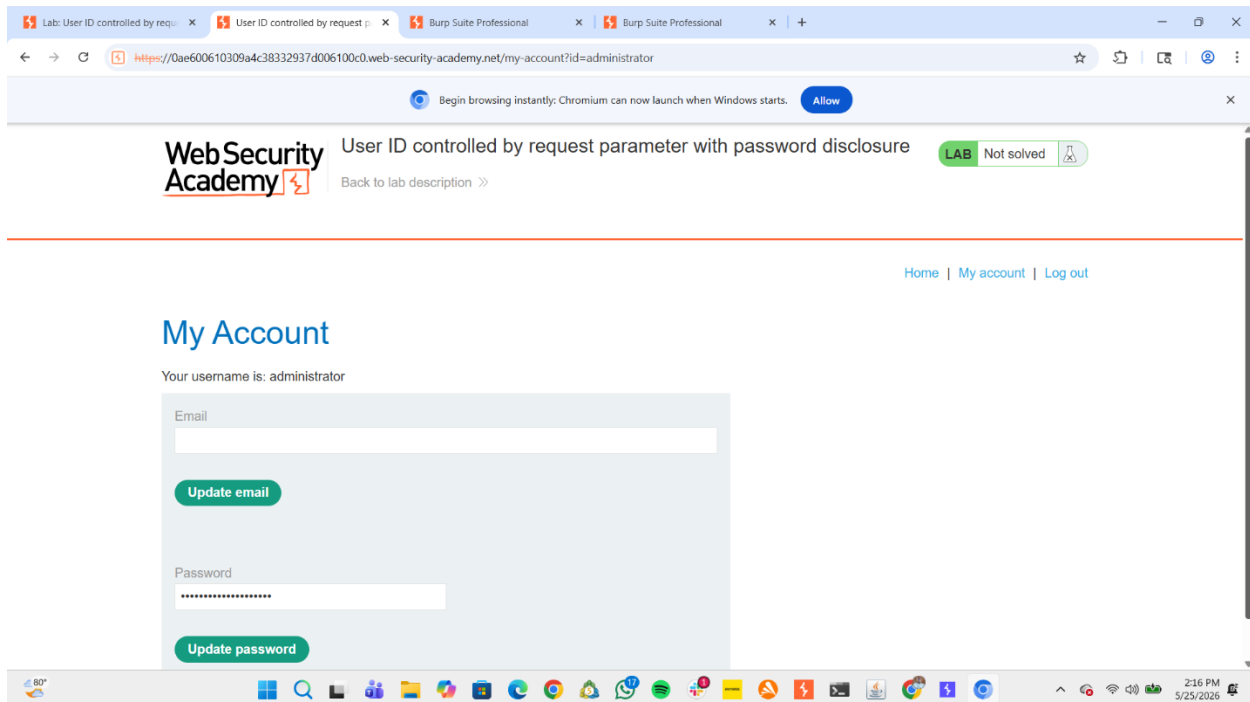
More specifically:

User ID Controlled by Request Parameter

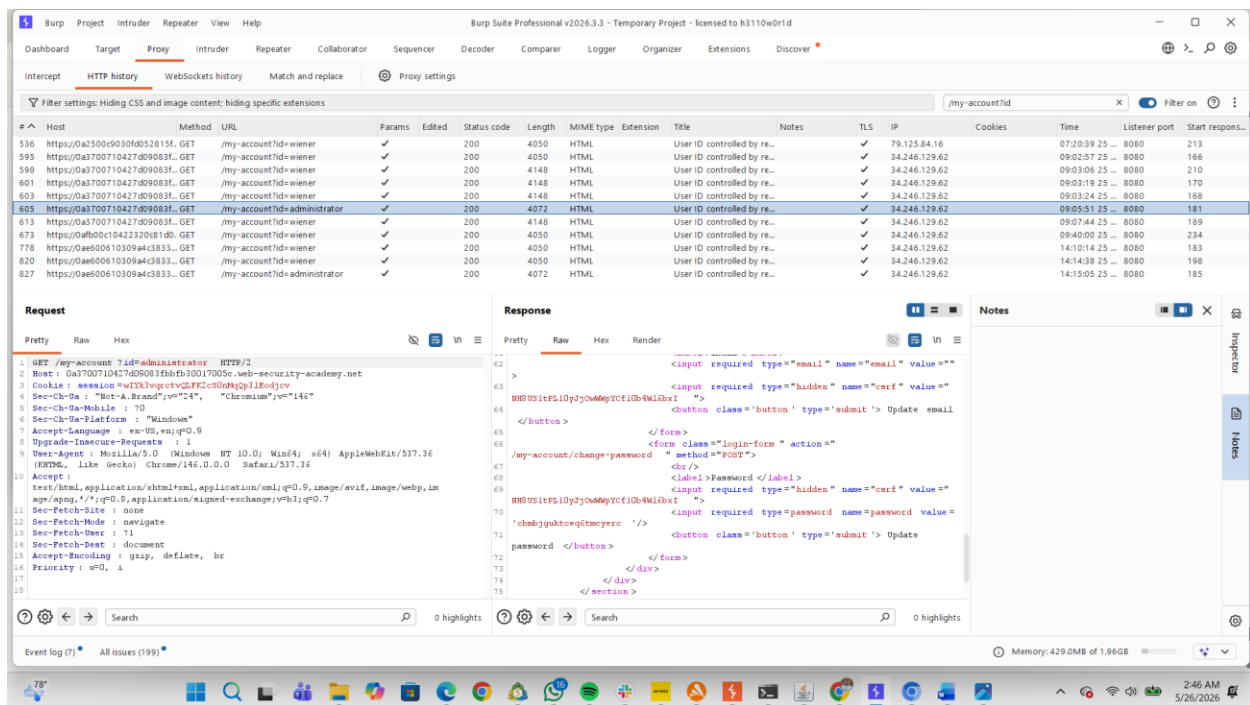
The application trusted user input instead of validating ownership of the requested resource.



Task 3: Proof of Concept (PoC)



This the administrator account page accessed by modifying the id parameter.



The vulnerability exposed sensitive information belonging to other users, including privileged administrator accounts.

An attacker could use this weakness to obtain confidential user information, compromise administrator accounts, and potentially gain full control over application functionality.

Because account identifiers were predictable and user-controlled, all users of the application could be affected.

This vulnerability could also result in privacy violations and non-compliance with data protection regulations.

Task 5: Remediation Plan

Issue	Recommended Fix	Security Benefit
User-controlled account identifier	Implement server-side authorization checks	Prevents unauthorized account access
Predictable user identifiers	Use indirect references or random identifiers	Reduces enumeration attacks
Password disclosure in responses	Never expose passwords in HTML responses	Protects sensitive credentials
Weak access control validation	Validate session ownership before serving data	Ensures users access only authorized resources
Excessive trust in client input	Enforce access decisions on the server side	Prevents parameter manipulation attacks

6: References

- (i)** Portswigger Web Security Academy
- (ii)** Burp suite Documentation
- (iii)** OWASP Broken Access Control

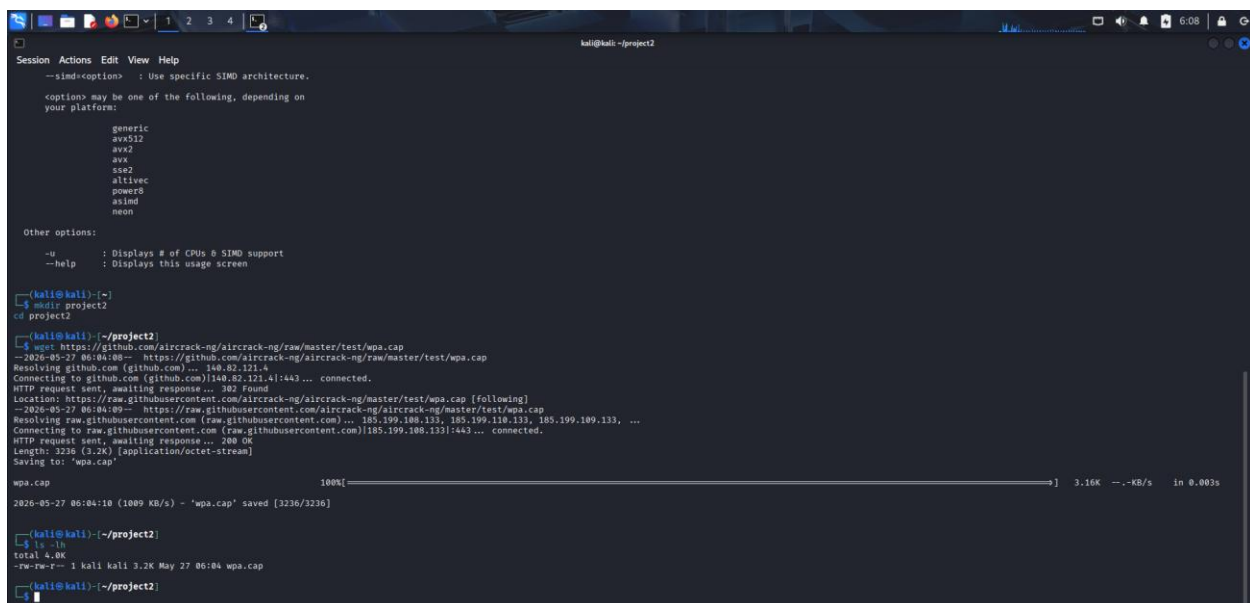
Project 2 – WPA/WPA2 Handshake Analysis

Objective

To verify the presence of WPA/WPA2 handshake traffic in a packet capture and determine the associated Wi-Fi password through authorized password auditing.

Capture File Acquisition

“git clone <https://github.com/aircrack-ng/aircrack-ng.git>”



```
kali@kali: ~/project2
--simd<option> : Use specific SIMD architecture.
<option> may be one of the following, depending on
your platform:
    generic
    armv5t2
    armv6
    armv7
    altivec
    powerpc
    neon

Other options:
    -u : Displays # of CPUs & SIMD support
    --help : Displays this usage screen

[kali@kali] (~)
$ mkdir project2
$ cd project2

[kali@kali] (~project2)
$ wget https://github.com/aircrack-ng/aircrack-ng/raw/master/test/wpa.cap
--2026-05-27 06:04:08-- https://github.com/aircrack-ng/aircrack-ng/raw/master/test/wpa.cap
Resolving github.com (github.com)... 148.82.121.4
Connecting to github.com (github.com):443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/aircrack-ng/aircrack-ng/master/test/wpa.cap [following]
--2026-05-27 06:04:09-- https://raw.githubusercontent.com/aircrack-ng/aircrack-ng/master/test/wpa.cap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.110.133, 185.199.109.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com):443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3236 (3.2K) [application/octet-stream]
Saving to: 'wpa.cap'

wpa.cap                               100%[=====] 3.16K --.-KB/s in 0.003s

2026-05-27 06:04:10 (1009 KB/s) - 'wpa.cap' saved [3236/3236]

[kali@kali] (~project2)
$ ls -ln
total 4.0K
-rw-rw-r-- 1 kali kali 3.2K May 27 06:04 wpa.cap

[kali@kali] (~project2)
```

Capture Validation

The file was validated using the following command:

```
kali@kali: ~/project2
$ wpa_cli project2
cd project2
kali@kali:~/project2$ $ wget https://github.com/aircrack-ng/aircrack-ng/raw/master/test/wpa.cap
--2026-05-27 06:04:10-- https://github.com/aircrack-ng/aircrack-ng/raw/master/test/wpa.cap
Resolving github.com (github.com)... 140.82.121.4
Connecting to github.com (github.com)[140.82.121.4]:443... connected.
HTTP request sent, awaiting response... 302 found
Location: https://raw.githubusercontent.com/aircrack-ng/aircrack-ng/master/test/wpa.cap [following]
--2026-05-27 06:04:10-- https://raw.githubusercontent.com/aircrack-ng/aircrack-ng/master/test/wpa.cap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 155.199.108.133, 155.199.110.133, 155.199.109.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)[155.199.108.133]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3236 (3.2K) [application/octet-stream]
Saving to: 'wpa.cap'

wpa.cap                               100%[=====] 3.16K --.-KB/s  in 0.003s

2026-05-27 06:04:10 (1009 KB/s) - 'wpa.cap' saved [3236/3236]

kali@kali:~/project2$ $ ls -lh
total 4.0K
-rw-rw-r-- 1 kali kali 3.2K May 27 06:04 wpa.cap

kali@kali:~/project2$ $ file wpa.cap
wpa.cap: pcap capture file, microsecond ts (little-endian) - version 2.4 (802.11 with Prism header, capture length 65535)

kali@kali:~/project2$
```

WPA Handshake Verification

The capture was analyzed using Aircrack-ng.

A WPA handshake was successfully identified.

```
kali@kali:~/project2
$ wpa_cli project2
cd project2
kali@kali:~/project2$ $ wget https://github.com/aircrack-ng/aircrack-ng/raw/master/test/wpa.cap
--2026-05-27 06:04:10-- https://github.com/aircrack-ng/aircrack-ng/raw/master/test/wpa.cap
Resolving github.com (github.com)... 140.82.121.4
Connecting to github.com (github.com)[140.82.121.4]:443... connected.
HTTP request sent, awaiting response... 302 found
Location: https://raw.githubusercontent.com/aircrack-ng/aircrack-ng/master/test/wpa.cap [following]
--2026-05-27 06:04:10-- https://raw.githubusercontent.com/aircrack-ng/aircrack-ng/master/test/wpa.cap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 155.199.108.133, 155.199.110.133, 155.199.109.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)[155.199.108.133]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3236 (3.2K) [application/octet-stream]
Saving to: 'wpa.cap'

wpa.cap                               100%[=====] 3.16K --.-KB/s  in 0.003s

2026-05-27 06:04:10 (1009 KB/s) - 'wpa.cap' saved [3236/3236]

kali@kali:~/project2$ $ ls -lh
total 4.0K
-rw-rw-r-- 1 kali kali 3.2K May 27 06:04 wpa.cap

kali@kali:~/project2$ $ file wpa.cap
wpa.cap: pcap capture file, microsecond ts (little-endian) - version 2.4 (802.11 with Prism header, capture length 65535)

kali@kali:~/project2$ $ capinfos wpa.cap
File name: wpa.cap
File type: Wireshark/tcpdump/... - pcap
File encapsulation: IEEE 802.11 plus Prism II monitor mode radio header
File timestamp precision: microseconds (6)
Packet size limit: file hdr: 65535 bytes
Number of packets: 13
File size: 3,236 bytes
Data size: 3,004 bytes
Capture duration: 0.078607 seconds
Earliest packet time: 2005-05-10 06:01:06.689737
Latest packet time: 2005-05-10 06:01:06.688344
Data byte rate: 38 kBps
Data bit rate: 305 kbps
Average packet size: 231.98 bytes
Average packet rate: 165 packets/s
SHA256: f1ec01802342a87f5f8d0b03b0d0e5c26b0b5f4d1b31acf0b1972f1b4f20e
SHA1: f5b157a3af40ab368ceaccbf504d7dbf91ad5
Strict time order: True
Number of interfaces in file: 1
Interface 00 info: Encapsulation = IEEE 802.11 plus Prism II monitor mode radio header (21 - ieee-802-11-prism)
Capture length = 65535
Time precision = microseconds (6)
Time ticks per second = 1000000
Number of stat entries = 0
Number of packets = 13

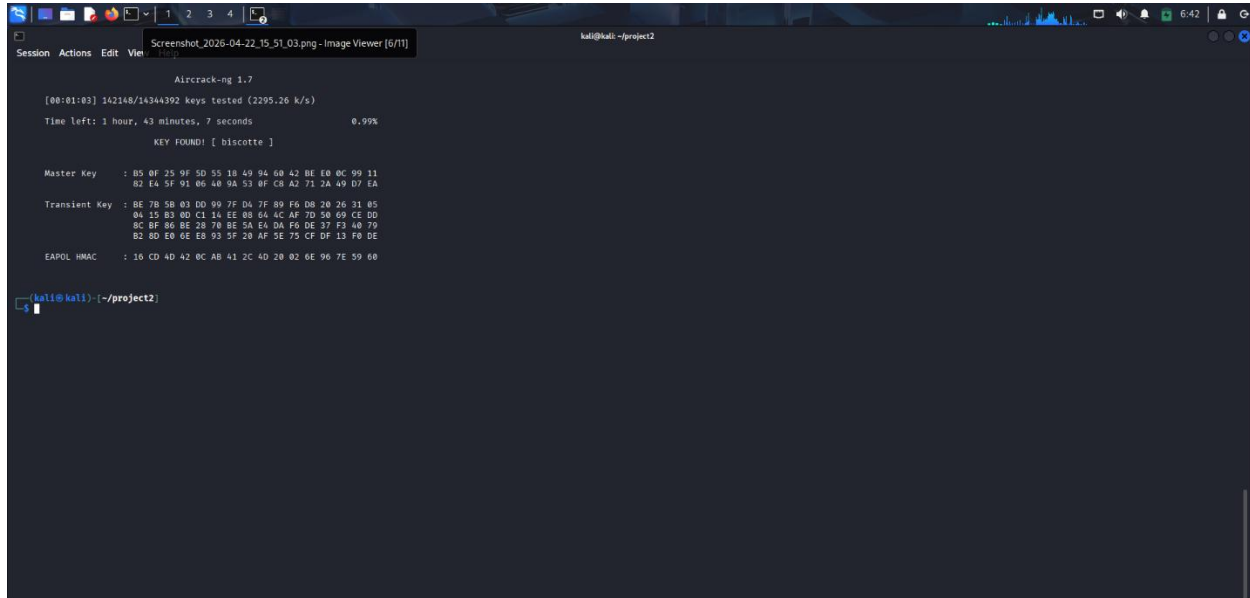
kali@kali:~/project2$
```

Password Recovery

A dictionary-based password audit was performed against the handshake.

The password was successfully recovered.

Recovered Password: [biscotte]



```
Aircrack-ng 1.7
[00:01:03] 142148/14344392 keys tested (2295.26 k/s)
Time left: 1 hour, 43 minutes, 7 seconds      0.99%
KEY FOUND! [ biscotte ]

Master Key   : 05 0F 25 9F 5D 55 10 49 94 60 42 BE E0 0C 99 11
              02 E4 5F 91 06 40 5A 53 9F C0 A2 71 2A 49 D7 EA
Transient Key : 0E 70 50 03 D0 99 7F D4 7F 89 F0 D0 20 26 31 05
              04 15 83 0D C1 14 EE 00 04 4C AF 7D 50 60 C2 00
              0C BF 86 BE 28 70 BE 5A EA DA F6 DE 37 F3 40 79
              02 0D E0 0E E0 93 5F 20 AF 5E 75 CF DF 13 F0 DE
EAPOL HMAC   : 16 CD 4D 42 0C AB 41 2C 4D 20 02 6E 96 7E 59 60

kali@kali: ~/project2
```

Why the Password Was Recoverable

The password was recoverable because it was weak enough to be contained within the testing wordlist used during analysis. WPA/WPA2 allows offline password verification against captured handshakes, making weak passwords vulnerable to dictionary attacks.

WPA3 Mitigation

WPA3 improves security by introducing SAE authentication, which provides stronger resistance to offline password guessing attacks and significantly reduces the effectiveness of handshake-based password auditing.

Conclusion

The packet capture contained a valid WPA/WPA2 handshake. Analysis confirmed that the Wi-Fi password could be recovered through authorized offline password auditing. This demonstrates the importance of strong passwords and modern wireless security standards such as WPA3.

When you get to the Aircrack-ng step and run the commands, send me the outputs or screenshots. I can then help you complete the report with the exact recovered password and tailor it to your findings.

Project 3: SQL Injection at Medium Security Level (DVWA)

Part 1: Environment Verification

Objective

The objective of this assessment was to determine whether SQL Injection vulnerabilities remain exploitable in DVWA when configured at the Medium security level.

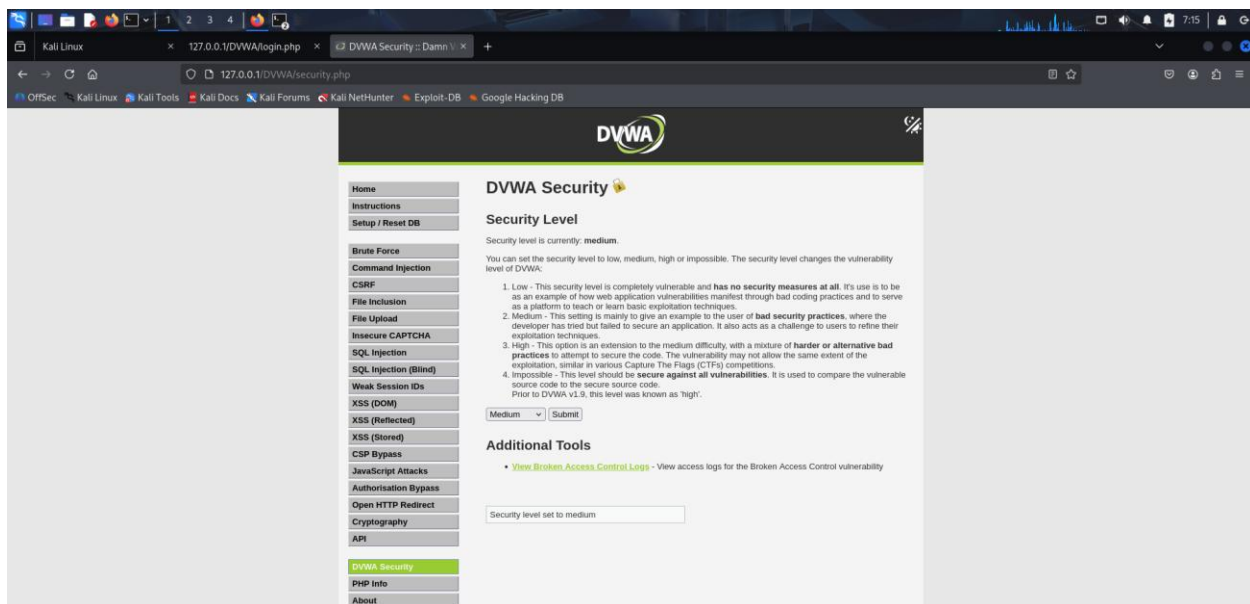
Environment Setup

- Application: Damn Vulnerable Web Application (DVWA)
- Platform: Kali Linux
- Database: MariaDB/MySQL
- Security Level: Medium
- Testing Scope: Local DVWA Instance Only

Verification

The DVWA application was successfully installed and configured on the local machine. The database was created and initialized using the DVWA setup page.

The security level was verified and set to **Medium** through the DVWA Security configuration page.



The SQL Injection module was identified under the Vulnerabilities menu.

Part 2: Understanding the Defense

What Basic Protection Does DVWA Medium Add Compared to Low?

DVWA Medium introduces additional restrictions compared to Low security by replacing the free-text input field with a drop-down list of predefined user IDs. This limits direct user input through the browser interface and makes basic SQL injection attempts more difficult.

Why Do Simple SQL Injection Attempts Fail?

Simple SQL injection attempts are harder to perform because users cannot directly type arbitrary values into the input field. The application restricts selections to predefined user IDs through the drop-down menu.

What Common Developer Mistake Still Allows SQL Injection?

Although the user interface restricts input, the application still relies on user-supplied data when constructing database queries. If an attacker modifies requests before they reach the server, SQL injection may still be possible. The root cause is the failure to use parameterized queries and prepared statements.

Part 3: Exploitation (Discovery Phase)

Initial Testing

The SQL Injection module was tested using valid user IDs.

Test 1

Input:

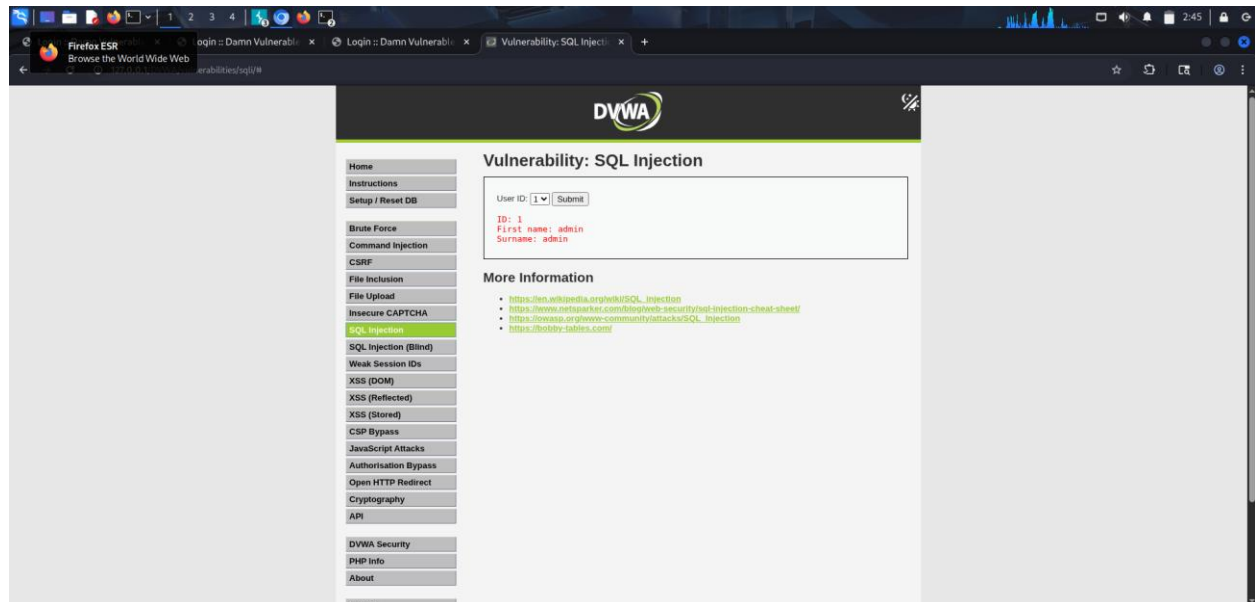
User ID = 1

Output:

ID: 1

First Name: admin

Surname: admin



Test 2

Input:

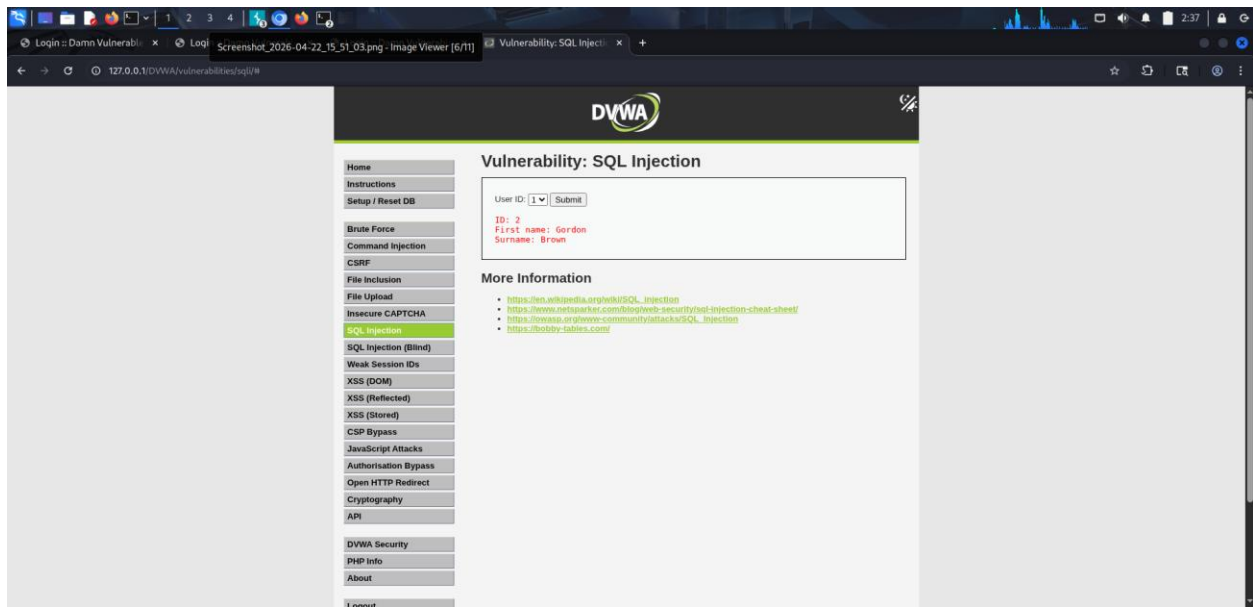
User ID = 2

Output:

ID: 2

First Name: Gordon

Surname: Brown



Test 3

Input:

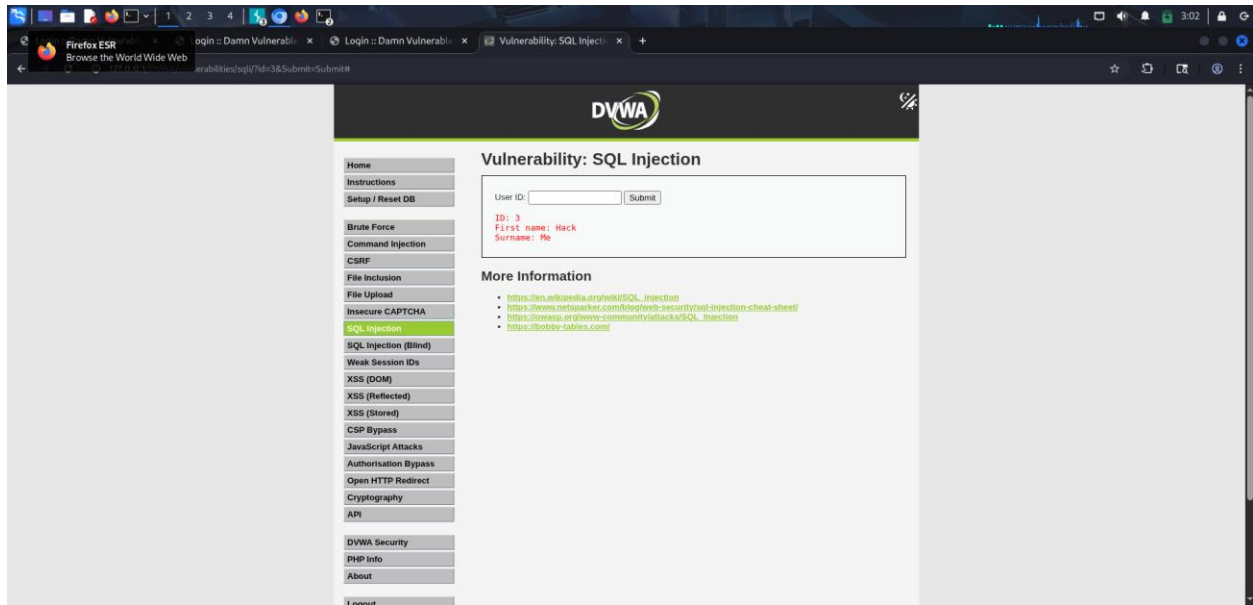
User ID = 3

Output:

ID: 3

First Name: Hack

Surname: Me



Analysis

The results demonstrate that the application accepts a user-controlled identifier and uses it to retrieve records from the database. Different IDs return different database records, indicating that user input directly influences backend database queries.

Vulnerable Request Identification

A request to the SQL Injection module was captured during testing:

```
GET /DVWA/vulnerabilities/sqli/ HTTP/1.1
Host: 127.0.0.1
Cookie: PHPSESSID=<session>; security=medium
```

The application processes a user-selected ID value and uses it to retrieve data from the database.

Why SQL Injection Remains Possible

Medium security primarily introduces client-side restrictions through the drop-down menu. However, these controls do not eliminate the underlying risk because attackers can modify requests using tools such as Burp Suite. Since the application does not use parameterized queries, the database remains vulnerable to manipulated input.

Part 4: Evidence of Exploitation

The application returned different user records when different User IDs were selected.

Evidence collected showed:

- User ID 1 returned the administrator account.
- User ID 2 returned Gordon Brown.
- User ID 3 returned Hack Me.

This demonstrates that the application relies on externally supplied input to determine which records are returned from the database.

Why the Bypass Works

The application trusts user-controlled values and incorporates them into database queries without using secure query mechanisms. Client-side restrictions alone are insufficient because attackers can modify requests outside the browser interface.

Part 5: Defense and Remediation

Why Medium-Level Protection Is Insufficient

Medium-level protection only limits input through the user interface. It does not prevent attackers from manipulating requests using interception tools or custom HTTP requests.

Exact Coding Mistake

The application constructs SQL queries using user-controlled input rather than using prepared statements and parameterized queries.

Recommended Fixes

Issue	Fix	Security Benefit
User-controlled SQL input	Use prepared statements	Prevents SQL injection
Dynamic query construction	Use parameterized queries	Separates SQL code from user data
Weak input handling	Validate and sanitize input server-side	Reduces attack surface
Excessive database permissions	Apply least-privilege access	Limits impact of compromise

Secure Development Recommendations

1. Use prepared statements for all database queries.
2. Implement parameterized queries instead of string concatenation.

3. Validate input on the server side.
 4. Apply least-privilege permissions to database accounts.
 5. Log and monitor suspicious query activity.
-

Conclusion

Testing confirmed that the DVWA SQL Injection module remains vulnerable at Medium security level. While additional controls were introduced compared to Low security, these protections do not eliminate the underlying vulnerability. Proper remediation requires prepared statements, parameterized queries, and secure server-side validation practices.

References

- DVWA Official Documentation
- OWASP SQL Injection Guide
- Burp Suite Community Edition
- MariaDB Documentation